

讓 Agent Skills 介入你的軟體開發的工作流

講師：Gelis





Gelis 程式設計訓練營
軟體開發之路

GitHub Copilot

Claude



GitHub Copilot ChatGPT

吳俊毅(Gelis)



- 顧問資歷：
  - 鼎基資訊電腦 - Claude Code 訓練講師
  - 柏瑞投信 - GitHub Copilot 代理人開發導入顧問
  - 飛騰雲端 - GitHub Copilot 顧問訓練講師
  - 新安東京海上 - Redis Cache 導入與執行顧問、系統架構顧問
  - 新光產險 - Redis Cache 導入、架構設計顧問
  - 新竹科學園區旺宏電子 - .NET 6 系統升級顧問、企業框架設計顧問
  - 高雄一卡通 - Unit Test 單元測試導入顧問
  - 玉山銀行數金處 - Unit Test 單元測試導入
  - 裕隆汽車 – 前端框架 Vue 授課導入顧問
- 講師資歷：
  - PCHome 導入 Unit Test 教育訓練講師
  - GIGABYTE 技嘉 – ASP.NET Razor Pages 訓練講師
  - 資策會 - IBM 授權教育訓練中心 - UML 課程講師
  - 2015 集英信誠 - 與大師對談講師
  - Microsoft Visual Studio Everywhere 2017 台北場講師
  - .NET Conf Taiwan 2018/2019/2020/2021/2022/2023 講師
  - HiSKIO 平台講師
  - Study4.TW 社群講師

Microsoft® MVP Most Valuable Professional

微軟最有價值專家
MVP (2011~2017)



GitHub Copilot

Claude

Agenda

- 如何駕馭 AI 開發、AI 時代架構師的暖身運動
- 從『規格至上』來探討 Agent 驅動的規格分析與建模技巧
- 你覺得你是在做 Spec 驅動開發嗎？
  - 不只是將 prompts 變成可重複使用的命令
- 重新定義 AI 時代的 SDLC 軟體開發
- 用 Agent Skills 來定義系統開發與設計流程方法 (Methodology)
  - *定義 Sequence/Domain Modeling 與程式碼的 Skills*
- Lab：Skill 就只做 Skill 的事情
  - Skill 就是種技能：但又可讓 AI 專注在特定 Context 商業領域下

雖然 AI 做投影片很快

但是畫面雜訊太多!! 所以..我盡量少用

今天.. 除持非真的需要... 或者 AI 真的有表達到我想表達的內容 ~

先用我的方式表達重點

你認為 AI 寫的程式能不能
100% 被相信呢？

90%

80%

70%



我不是說 AI 寫程式不行，
其實 AI 真的非常厲害

但是.....



我不是說 AI 寫程式不行，
其實 AI 真的非常厲害

但是.....

我認為，
不讀 AI 寫的程式碼不單單一種決定、
而是一種『**能力**』！





GitHub Copilot Claude

下面有三種操作 AI 的方式：

(1). 一路交給 AI 來*完成所有事？？？*

分析需求 (Analysis) -> **解釋**與**設計需求** (Explain with Plan) -> 設計 (Design) -> **撰寫程式** (Coding) -> 測試 (Testing)

(2). 審核(介入)軟體開發的每一個階段？？？

分析需求 (Analysis) -> **解釋**與**設計需求** (Explain with Plan) -> 設計 (Design) -> **撰寫程式** (Coding) -> 測試 (Testing)

Review Review Review Review

(3). Vibe Coding 憑感覺寫程式

需求（prompts）只下提示詞 -> 反覆對話 Chat Window （prompts） -> 人工測試 (Testing)

你是哪一種呢？

如何駕馭 AI 開發、
AI 時代架構師的暖身運動



優秀的 Spec 並非單一指令，而是層次化的設計系統

在複雜系統中，我們需要將軟體架構的設計過程，轉化為一個有跡可循的 Spec 體系。這是一個從宏觀到微觀的拆解過程。

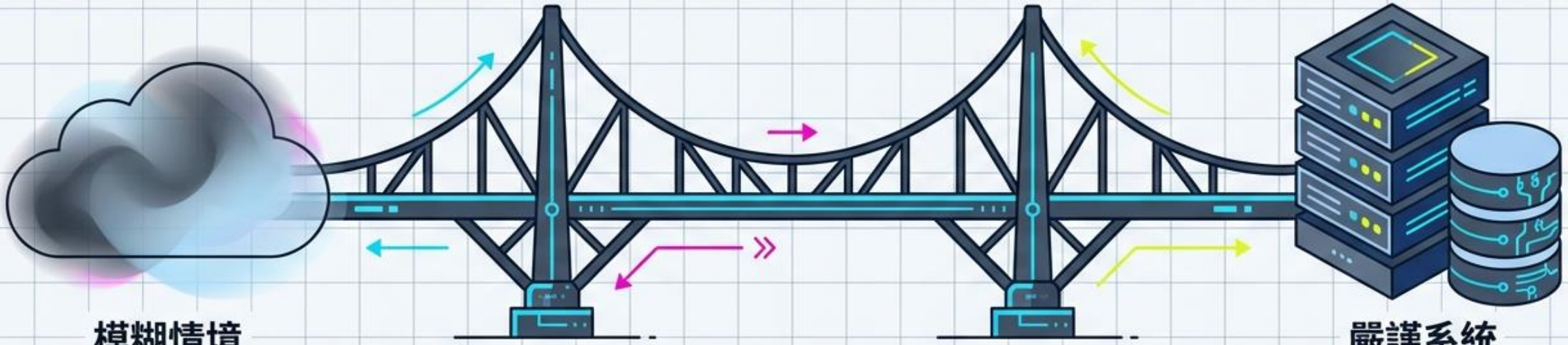


將軟體架構『最小化人力資源』的目標，應用到 Context 設計上。

從『規格至上』來探討 Agent 驅動的規格分析與建模技巧



AI 時代的軟體開發：
Agent 驅動的規格分析與建模術



[System Initiated] Bridging the gap from Scenario to System Architecture using Agent Skills.

你是在用 AI 進行建構，還是讓 AI 為你建構？

純人工建構 (Manual)



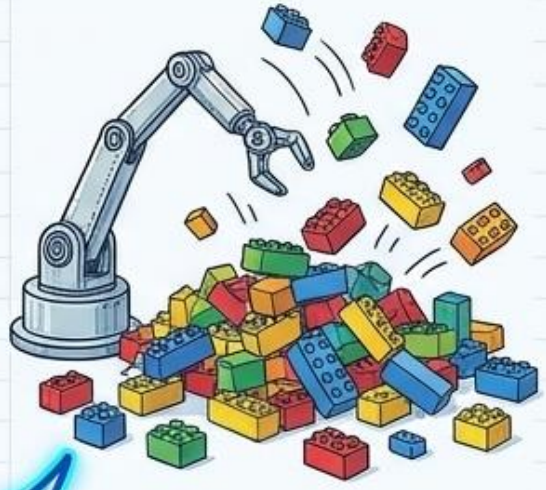
耗時費力，缺乏自動化動能。

Agent 驅動 (Control)



透過嚴謹的 Agent Skills 作為藍圖，將系統開發生命週期 (SDLC) 自動化且標準化。

無序的 Prompt (Chaos)



隨機對話，每次產生的規格與 UML 都不一致，難以維護。

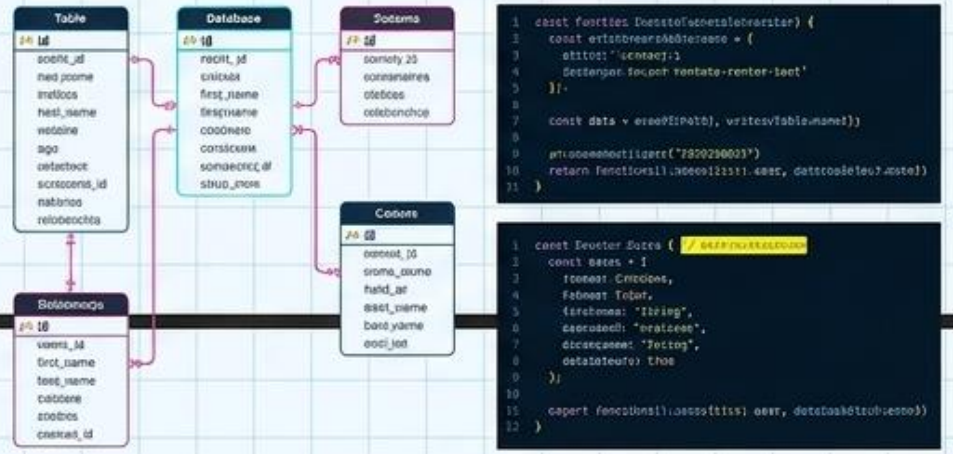
軟體架構設計與需求理解的交集

模糊商業需求 (Scenario)





精確系統規格 (System)



核心挑戰：如何將模糊的商業情境，精確轉換為 AI 能夠執行的工程提示詞 (Prompt)？

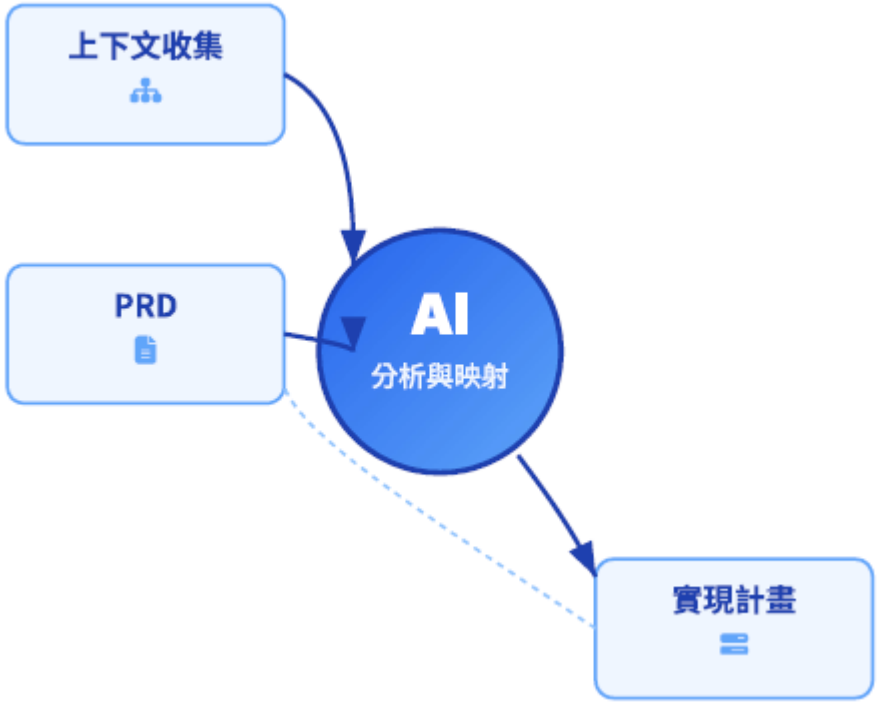
這不僅僅是 Prompting，這是系統分析與設計的重塑。

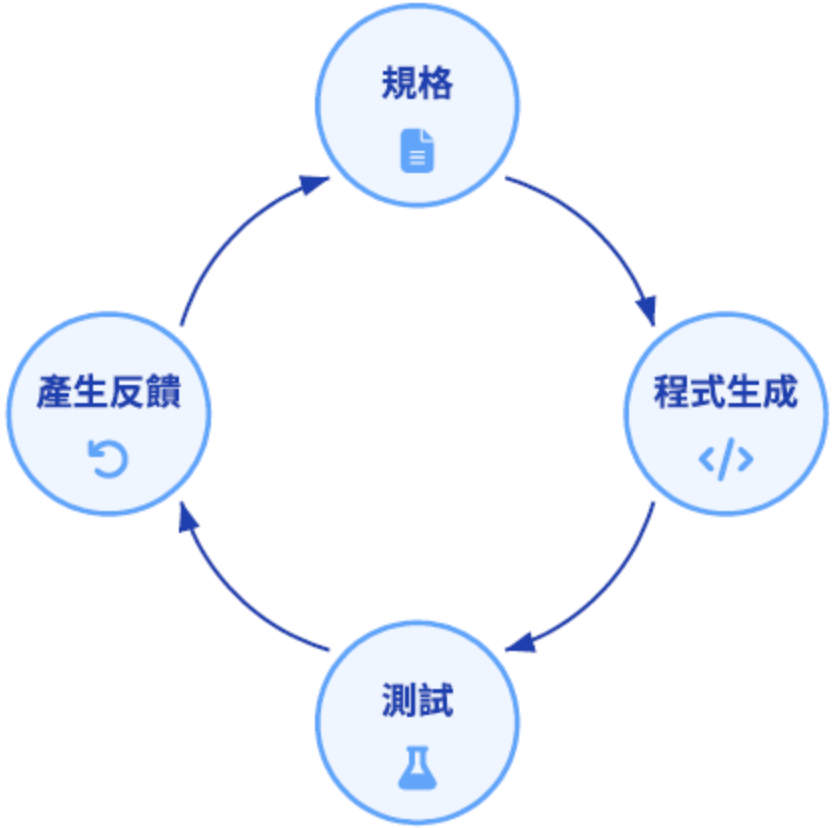
你覺得你是在做 Spec 驅動開發嗎？



SDD 的開發流程

- 將 prompts 需求上下文的收集 ➔ 到 PRD 實現計畫=自動化







GitHub Copilot Claude

Spec-Kit 提供的命令 (prompts)：

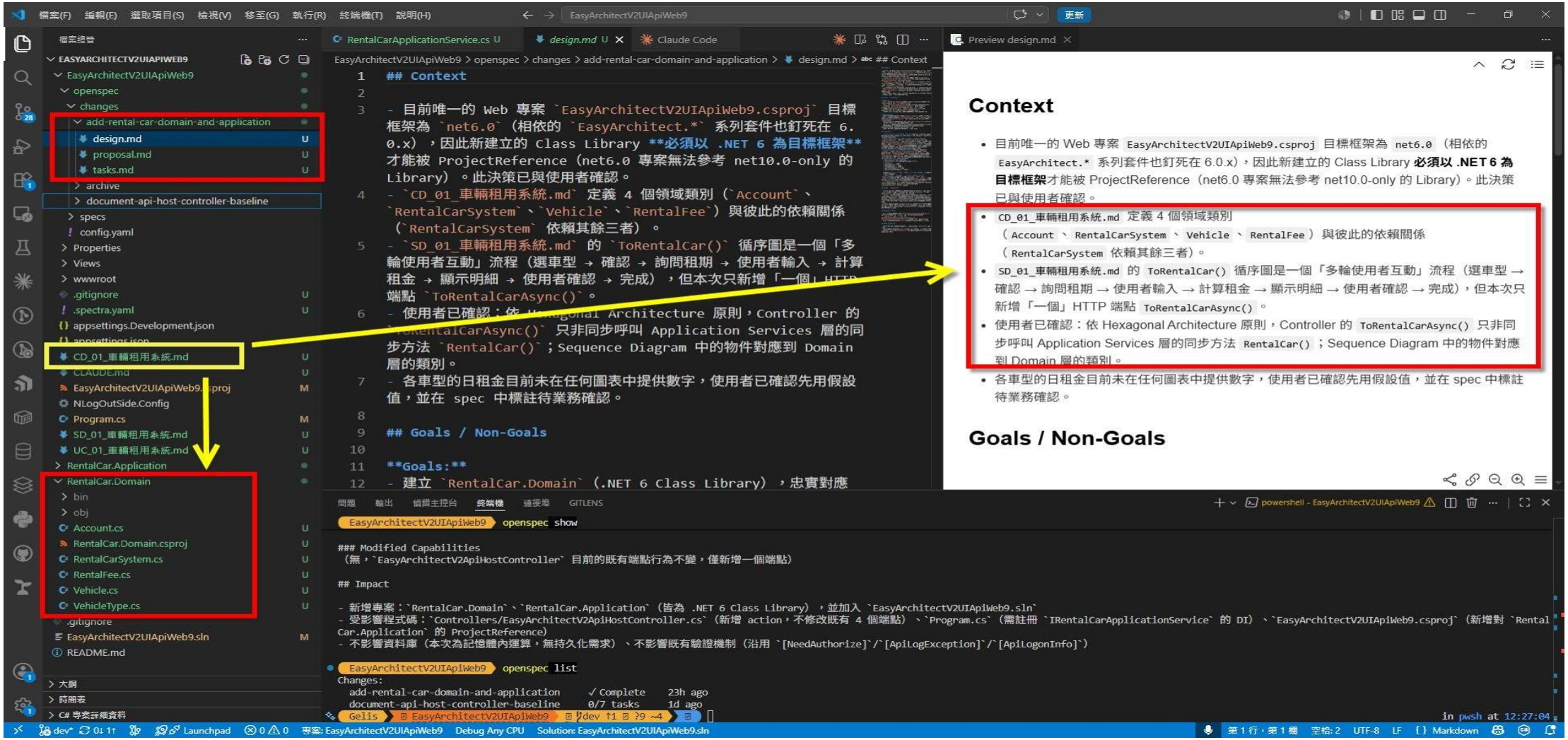
- Spec-Kit 提供了如下主要參數：

1. **/speckit.constitution** - 建立 or 定義專案核心要求與開發原則
2. **/speckit.specify** - 自動化規格的生成（What & Why）
3. **/speckit.plan** - 建立開發的實作計畫
4. **/speckit.tasks** - 產生可執行的開發工作項目
5. **/speckit.implement** - 執行上述產生的開發工作項目

---

6. **/speckit.clarify** - 需求釐清、可由 AI 分析當前規格中存在的潛在模糊性或衝突
7. **/speckit.checklist** - 進行品質把關審查，確保所有需求皆具備可測試性
8. **/speckit.analyze** - 分析缺漏的功能性需求、**tasks** 與 **plan** 是否存在不一致？

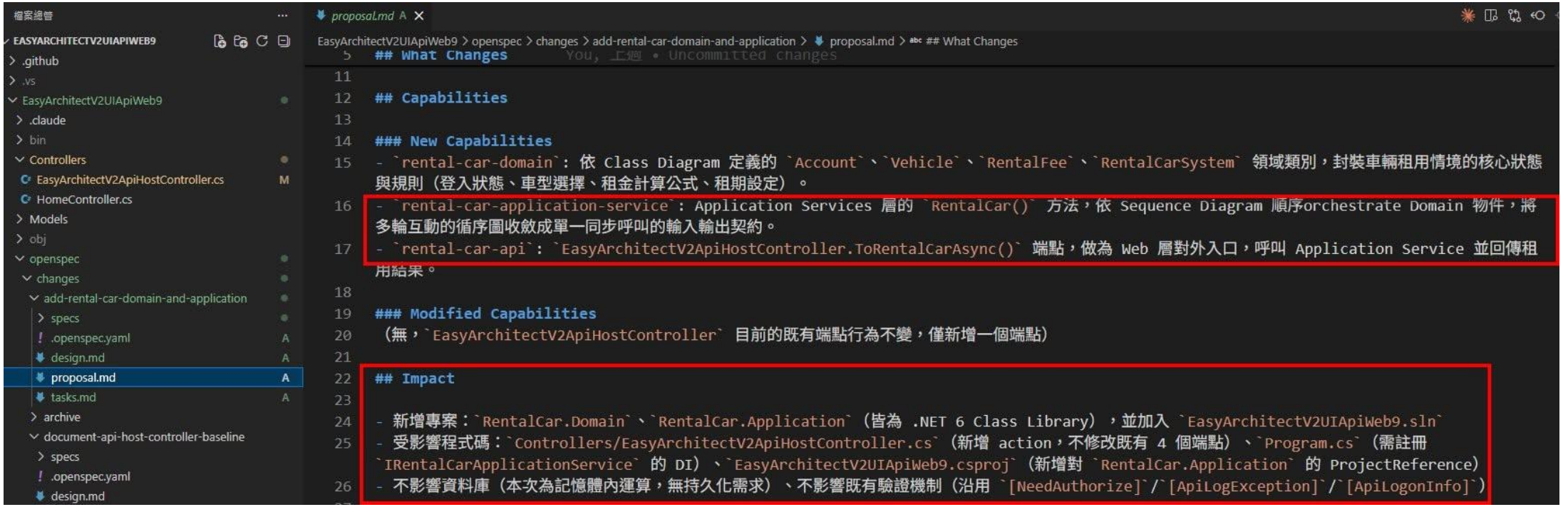
用 OpenSpec 來維護 Legacy System



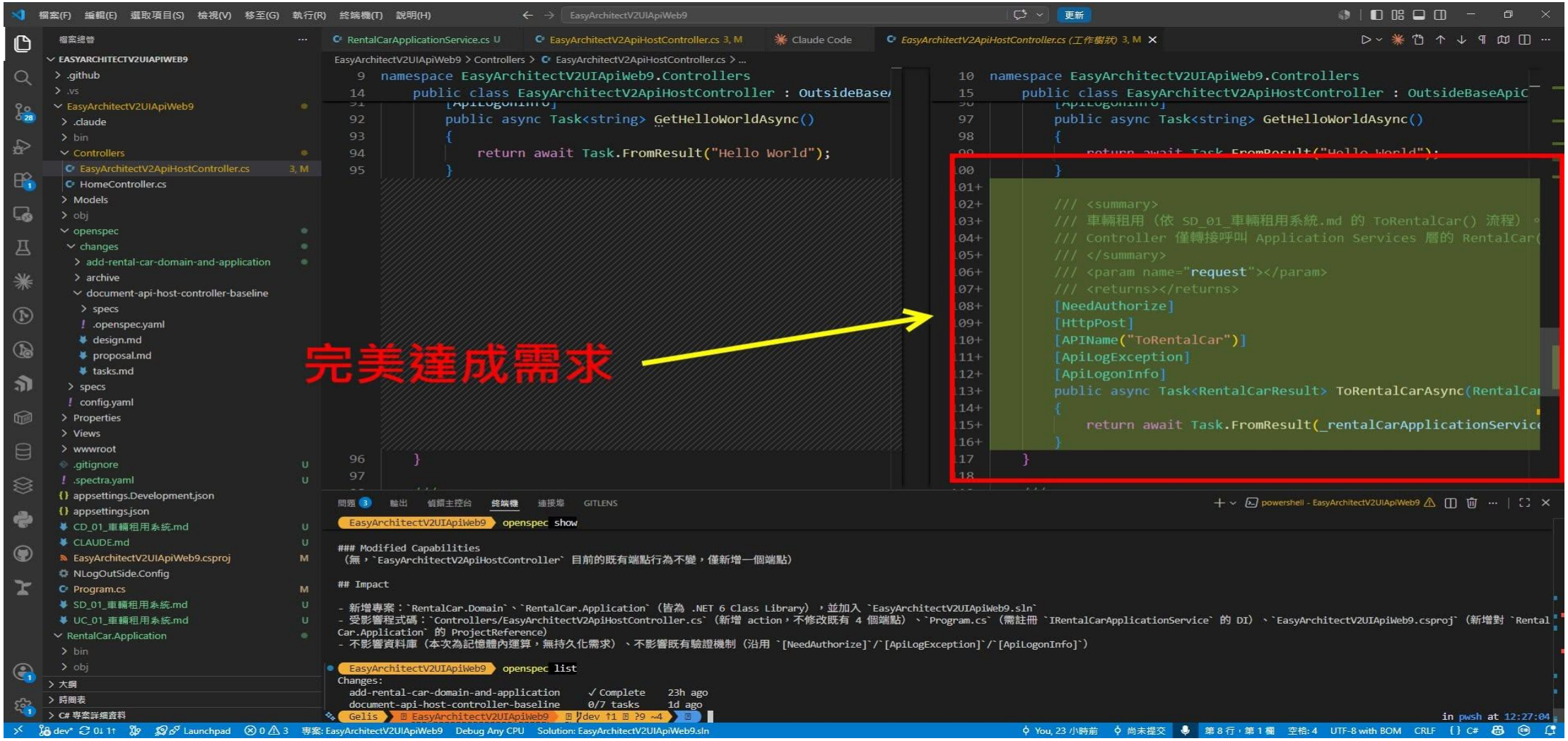


GitHub Copilot Claude

用 OpenSpec 來維護 Legacy System(2)

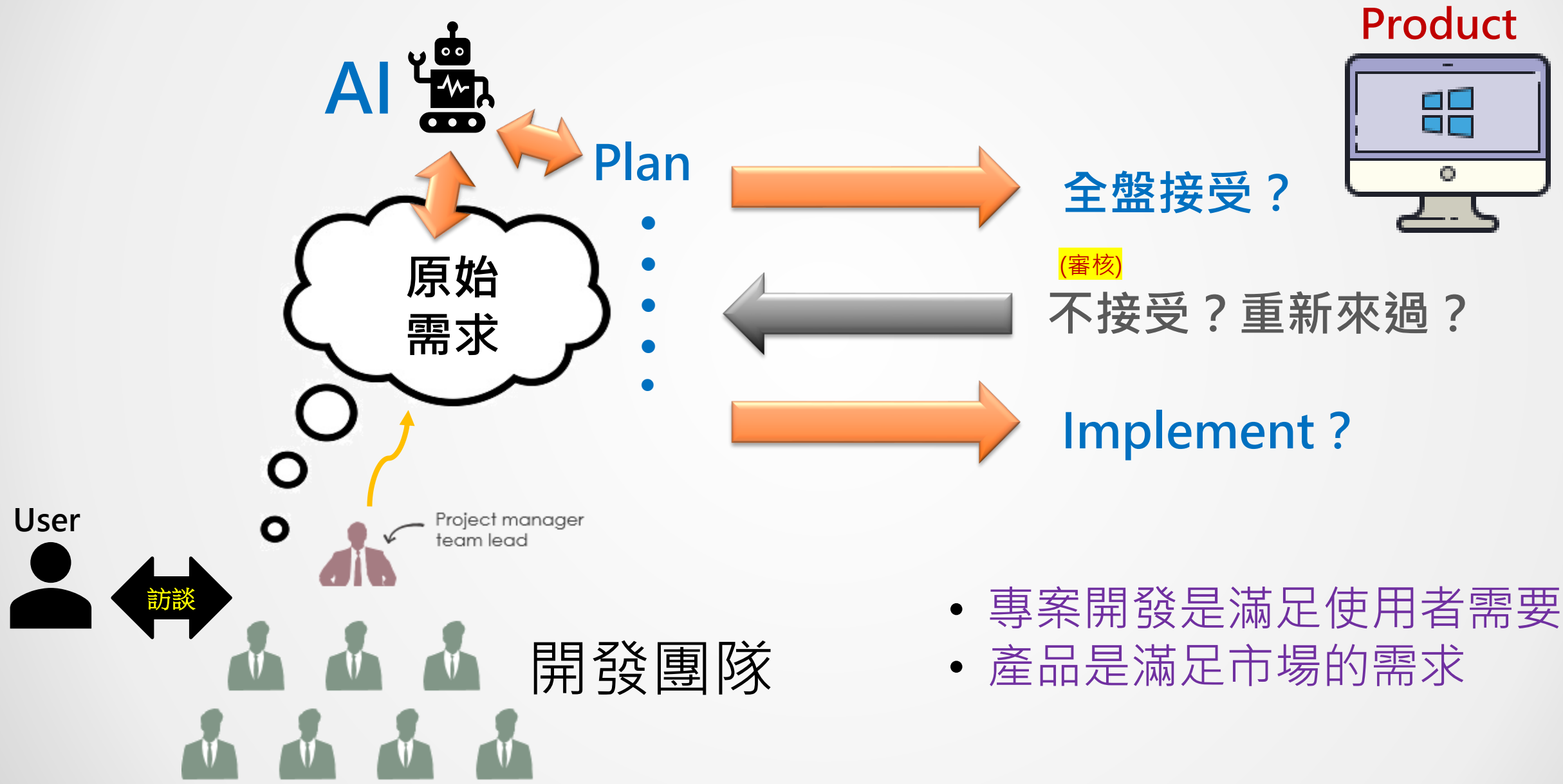


用 OpenSpec 來維護 Legacy System(3)



但文件 (Spec) 量能不能再更少一點？

我們來思考一個問題...



- 專案開發是滿足使用者需要
- 產品是滿足市場的需求

GitHub Copilot
Claude

Welcome
- Introduction
- Machine Learning
- AI Agents
- Enterprise AI
- Responsible AI
- AI for code
- Chatbots
- AI platforms
- Artificial general intelligence (AGI)
- Prompt engineering
- AI use cases
- Industry applications

The AI-DLC: The AI-driven development lifecycle

By Cole Stryker

The AI-DLC, explained

AI-DLC stands for "AI-driven development lifecycle," an augmentation of the software development lifecycle, or SDLC. It signifies a software development methodology completely built around AI, treating a large-language model (LLM), or group of models, as an active collaborator throughout every step of software development.

Coined by AWS's Raja SP in 2025 and described in his "AI-Driven Development Lifecycle (AI-DLC) Method Definition" whitepaper [x], the AI-DLC methodology is presented as a machine-human collaboration, as opposed to previous methods like Agile and Scrum, which were fundamentally human-centric. The author compares this shift to the replacement of horse-drawn carriages by the automobile. But while simply adding AI to Agile produces a "faster horse"—to borrow a metaphor often misattributed to Henry Ford, the AI-DLC represents a more fundamental evolution.

Whereas AI tools have been integrated into the SDLC in recent years, the AI-DLC represents a step of further integration of AI. In short, AI is not relegated to a mere code generation tool. It's not just a part of development, but central to it. Collaboration between AI and humans is continuous, and human

Guide

Start realizing ROI: A practical guide to agentic AI

Learn how to scale agentic AI for measurable ROI across your enterprise. This playbook outlines the top barriers that limit impact, how to effectively measure ROI and a practical framework to drive successful, enterprise-wide adoption.

Get the guide →

Next step in your AI journey

See watsonx Orchestrate in action →

Start your free trial →

AI



GitHub Copilot

Claude

SDD 與 AIDLC 兩者的關係

- SDD 是一種 AI 原生工程方法（AI-Native Engineering Approach）它工程師在專案裡用的「招式」（微觀）
  - 它規定了您必須準備 constitution.md、spec.md、plan.md 等文件
  - 不是 SDD 參考 AIDLC，而是 AIDLC 把 SDD 納入為它在「開發階段」的核心方法論
- AIDLC 是企業管理專案的「制度」（宏觀生命週期）
  - **AIDLC（AI-Driven Development Lifecycle）則是一整套架構營運模式**，它關注的是整個軟體工廠的流水線
  - 它涵蓋了從一開始的 AI 法律合規審查、AI 成本預算估算，到中間的程式碼生成（這一步就是採用 SDD），再到最後的 AI 自動監控與模型疊代
  - 例如 IBM 提出的 AI-DLC 框架，其實就是方法論建立了完美的 **Harness Engineering（駕馭環境）**，讓 SDD 的自動化驗證迴路（回饋控制）代替了人類的眼睛



GitHub Copilot

Claude

我的意思是.. 不是不要相信 AI

- AI 很能寫 Code 這是確定的!!!
- 但是，人類自己要先想清楚，AI 才會做出對的事情。



重新定義 AI 時代的 SDLC 軟體開發



但是之前有一件事，
一直很困擾著我！？

就是.. AI 每次寫的 Code 都不一樣？

同一個需求、同一個功能



但.. 為什麼

我又說我可以做到

不去 **Review** 我的 **AI** 的產出呢？

方法論
(Methodology)

「以方法論為導向的自主開發
（Methodology-Driven Autonomous Development）」



AI-native SDLC Playbook

- **Anthropic 公司** 曾發布過一份重磅指南：**《The AI-native SDLC Playbook》（AI 原生軟體開發生命週期劇本）**
  - 規格與政策優先（Policy applied while the spec is written）：
    - 在 AI 開始編碼前，公司的「方法論政策與制度」必須先被定死，而不是等代碼寫完才來 Review
  - 制度代碼化（Institutional knowledge becomes files the agent reads）：
    - 將人類工程師的「方法論、制度與架構知識」直接寫成檔案讓 AI Agent 閱讀，並將防護欄（Guardrails）直接作為程式碼運行，這與我的「用方法論預期 AI 代碼架構」的想法完全一致



GitHub Copilot

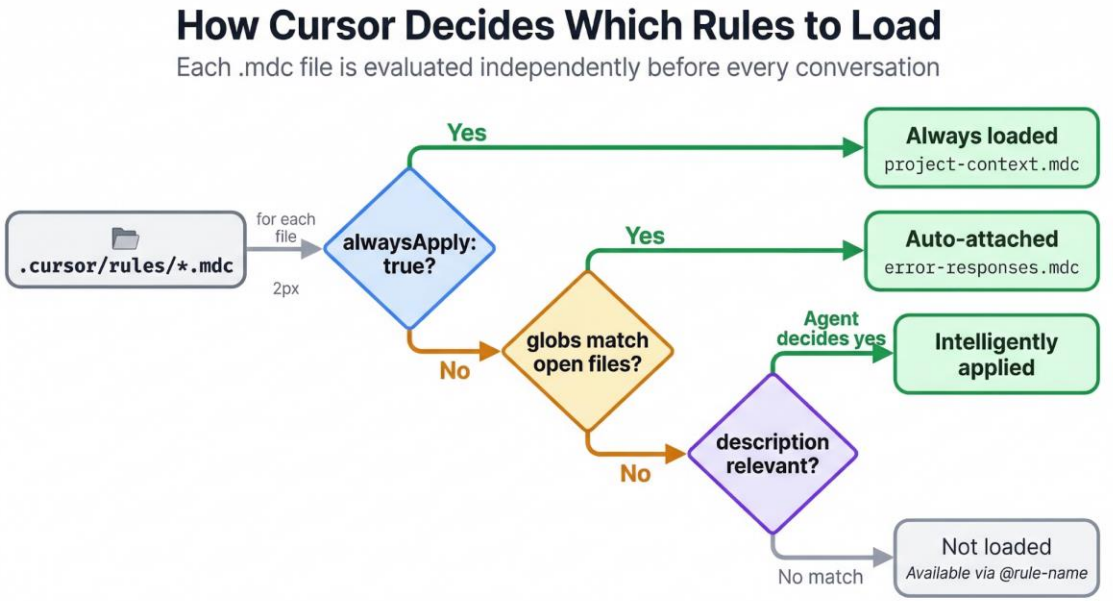
Claude

實踐 Harness Engineering

- 矽谷主流工具實踐：Cursor Rules (.mdc) 與 AGENTS.md
- 從 prompt 轉向到 AIDLC（AI驅動開發生命週期）
- 同時衍生出 Eval-Driven Development（評估驅動開發）的概念
- AI 行為與架構在開發前就定義

實踐 Harness Engineering

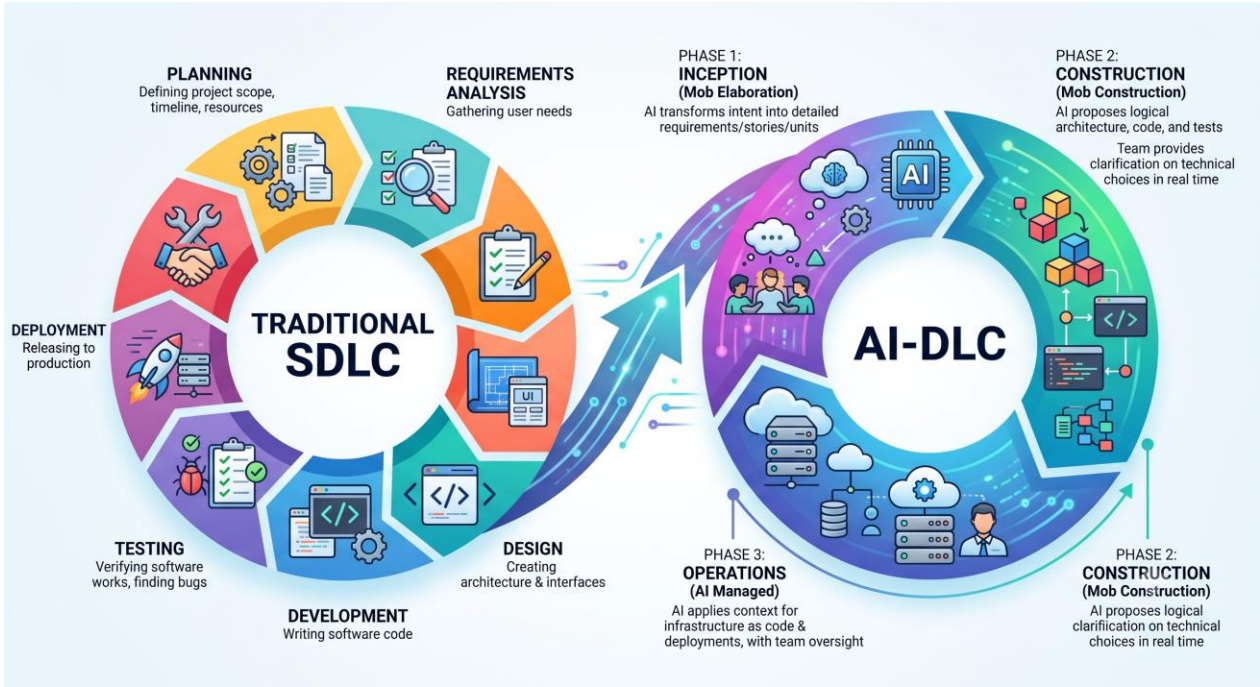
- 矽谷主流工具實踐：Cursor Rules (.mdc) 與 AGENTS.md
- 從 prompt 轉向到 AIDLC（AI驅動開發生命週期）
- 同時衍生出 Eval-Driven Development（評估驅動開發）的概念
- AI 行為與架構在開發前就定義





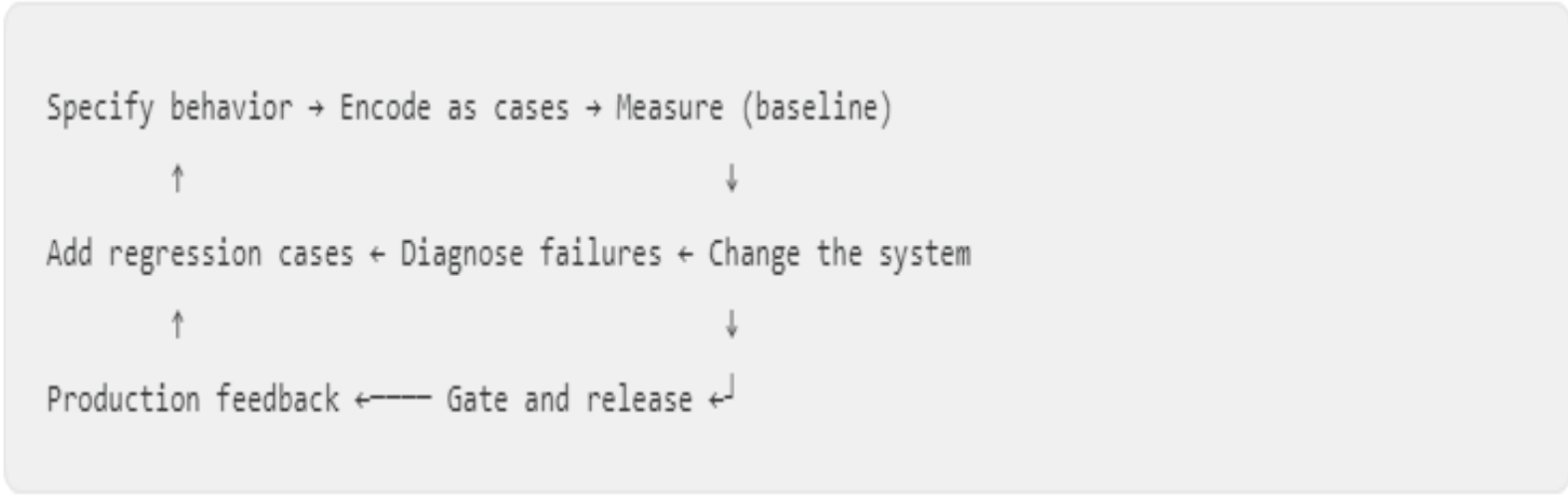
實踐 Harness Engineering

- 矽谷主流工具實踐：Cursor Rules (.mdc) 與 AGENTS.md
- 從 prompt 轉向到 AIDLC（AI驅動開發生命週期）
- 同時衍生出 Eval-Driven Development（評估驅動開發）的概念
- AI 行為與架構在開發前就定義



實踐 Harness Engineering

- 矽谷主流工具實踐：Cursor Rules (.mdc) 與 AGENTS.md
- 從 prompt 轉向到 AIDLC（AI驅動開發生命週期）
- 同時衍生出 Eval-Driven Development（評估驅動開發）的概念
- AI 行為與架構在開發前就定義



AI 時代的軟體開發：Agent 驅動的規格分析與建模術

說明如何透過結構化的 Prompt Engineering 與 Agent Skills，將商業情境精確地轉化為系統模型與軟體規格。

核心心法：AI 時代的 Prompt Engineering



SRP
單一職責分析法

確保 AI 每次只處理一個具體的事件，避免規格發數。



OOA/OOD
名詞分析法

透過拆解商業需求中的關鍵實體，屬滑系統核心模型。



UML
Use Case 收斂焦點

利用「主要事件流」定義系統邊界，有效收斂 AI 的注意力。

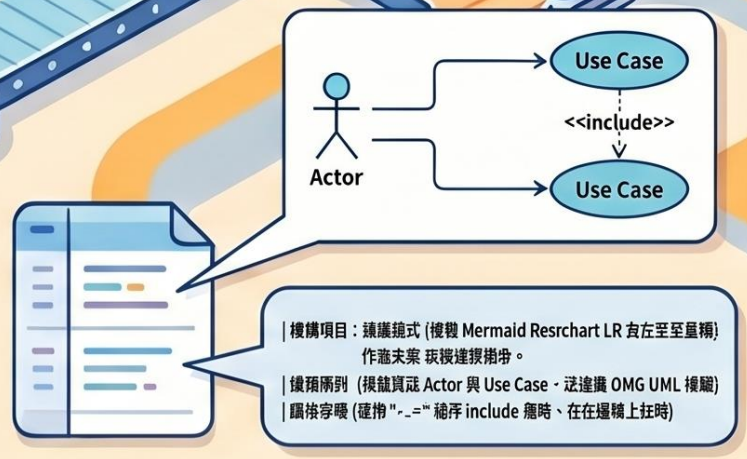
AI Agent

實踐流程：建立自動化 Agent Skills



建立 Prompts Libraries

將分析後可重櫬使用的命令僥存為齒式庫，作為系統緞護的根本。



定義專屬建模 Skills

嚴格規範輸出格式（如 Mermaid 語法），實現自動化繪製模型圖。

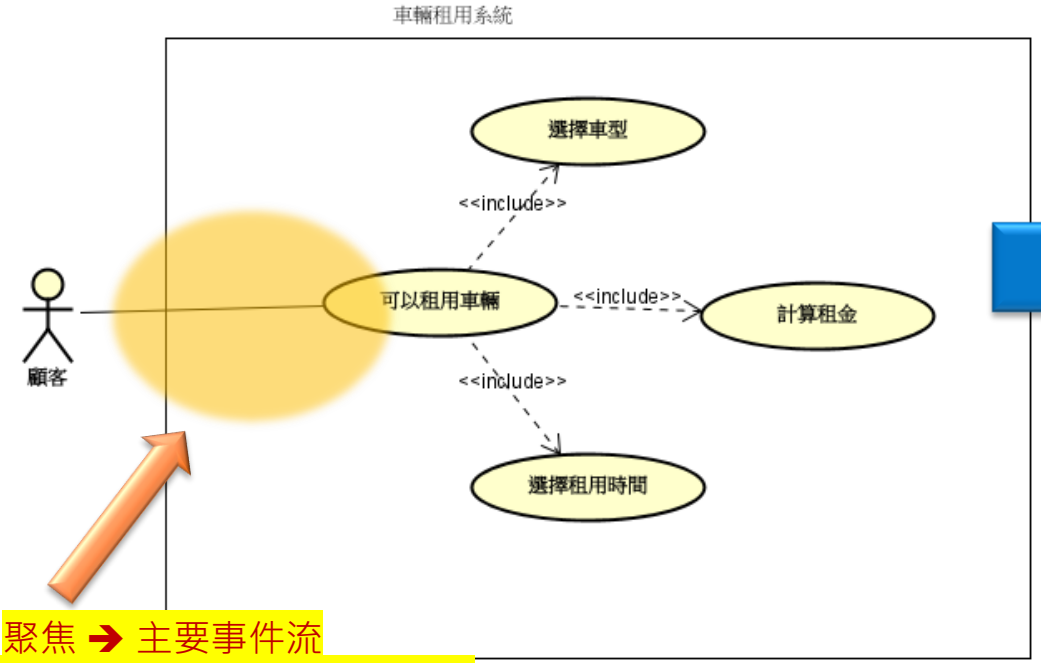


從廣泛到細節的淬鍊

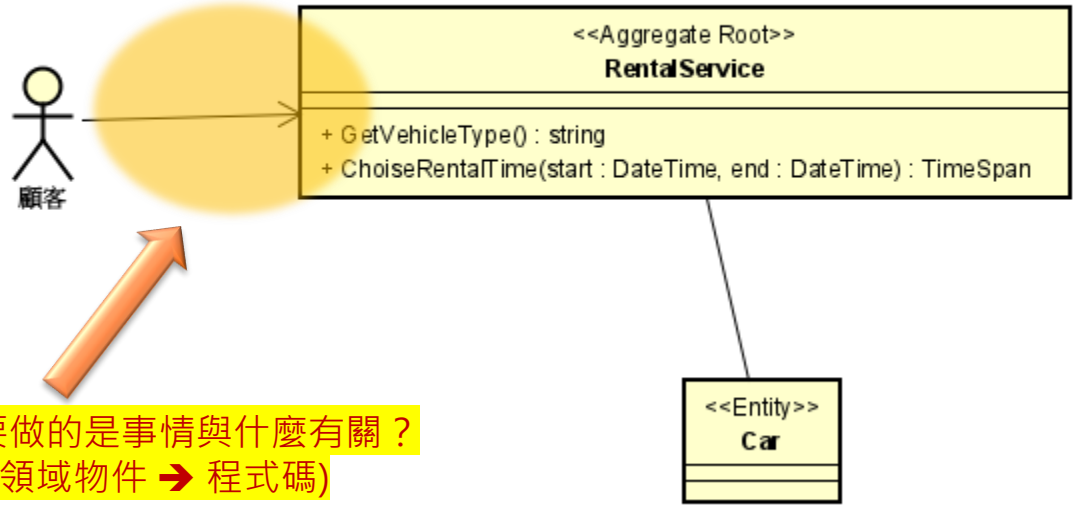
透過蹂步引導與歷史紀錄保持，確保規格產出足夠精確。

AI 其實非常熟悉原有的軟體開發方法論

- 以租車系統來說：
  - 聚焦 ➔ 主要事件流
  - Basic Flow or Main Success Scenario

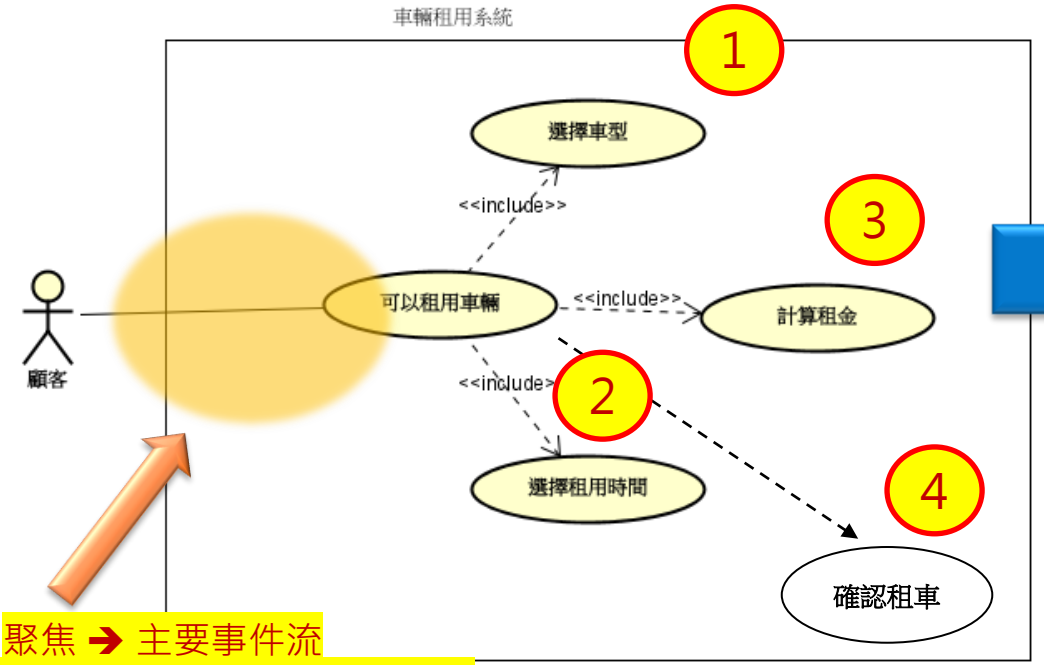


- 只聚焦在『顧客可進行租車』服務
  - 現在要做的是事情與什麼有關？
  - (領域物件 ➔ 程式碼)



AI 其實非常熟悉原有的軟體開發方法論

- 以租車系統來說：



聚焦 ➔ 主要事件流

Basic Flow or Main Success Scenario

-

```
RentalCarSerAppServices.cs M
Application9.RentalCar9 > RentalCarSerAppServices.cs > { } Application9.RentalCar9 > RentalCarSerAppServices > ToRental
 6  namespace Application9.RentalCar9
11      public class RentalCarSerAppServices
38          /// 租用車輛：選擇車型、選擇租用時間區間、計算租金，並確認租
39          /// </summary>
40          /// <param name="rentalCarDto">租車資料傳輸物件</param>
41          /// <returns>租用成功回傳 true，失敗回傳 false</returns>
            0 個參考
42          public bool T    lCar(RentalCarDto rentalCarDto)
43          {
44              // 1. 選擇車型
45              IVehicle vehicle = rentalCarDto.VehicleType == Veh
46                  ? new Car(rentalCarDto.Model) { CC = rentalCar
47                  : new RV(rentalCa    Model) { CC = rentalCarD
48
49              // 2. 選擇租用時間區間
50              int daysRented = vehicle.ChoiseRentalTime(rentalCa
51              if (daysRented <= 0)
52              {
53                  return false;
54              }
55
56              // 3. 計算租金
57              int rentalCost = vehicle.CalculateRentalCost(daysR
58              if (rentalCost <= 0)
59              {
60                  return false;
61              }
62
63              // 4. 確認租車
64              return _rentalCarRepository.AddCar(vehicle);
65          }
66      }
67  }
```

用 Agent Skills 來定義統開發與設計流程方法 (Methodology)



步驟二：解剖高精準度的建模 Skill

```
---
name: create-use-case-modeling
description: 請依照業務流程，以 Mermaid 建立 Use Case Diagram 圖形
---

<!-- Tip: Use /create-skill in chat to generate content with agent assistance -->

(1). 在 VSCode 中建立的 Mermaid 圖形必須以 ```mermaid`` 語法包裹起來，才能正確顯示圖形。
(2). 請使用 Flowchart LR 由左至右的方式來呈現 Use Case Diagram。
(3). 在圖形中清楚標註 Actor 與 Use Case 的名稱。
(4). 盡可能遵循 OMG 的 UML 規範、當使用到【包含】字眼時，明確標示 include。
(5). 在 Mermaid 中的 include 不要用 "<<include>>" 的方式來標示，而是用 "-.->" 的方式來標示，並在箭頭上註明 "include"。
(6). 請在 subgraph XXXXXXX系統 名稱後面加上 ["XXXXXXX系統(System Boundary)"]
(7). 允許在 Use Case 中使用特殊字元以顯示 ICON 美化
(8). 請在最下方描述與顯示 ## Use Case 說明
```

強制準則

強制遵循 OMG 規範，消滅 AI 自創語法與不合規的圖示。

修正渲染 Bug

避免使用雙括號導致 Mermaid 渲染失敗，精確指導 AI 標示 Include 關係。

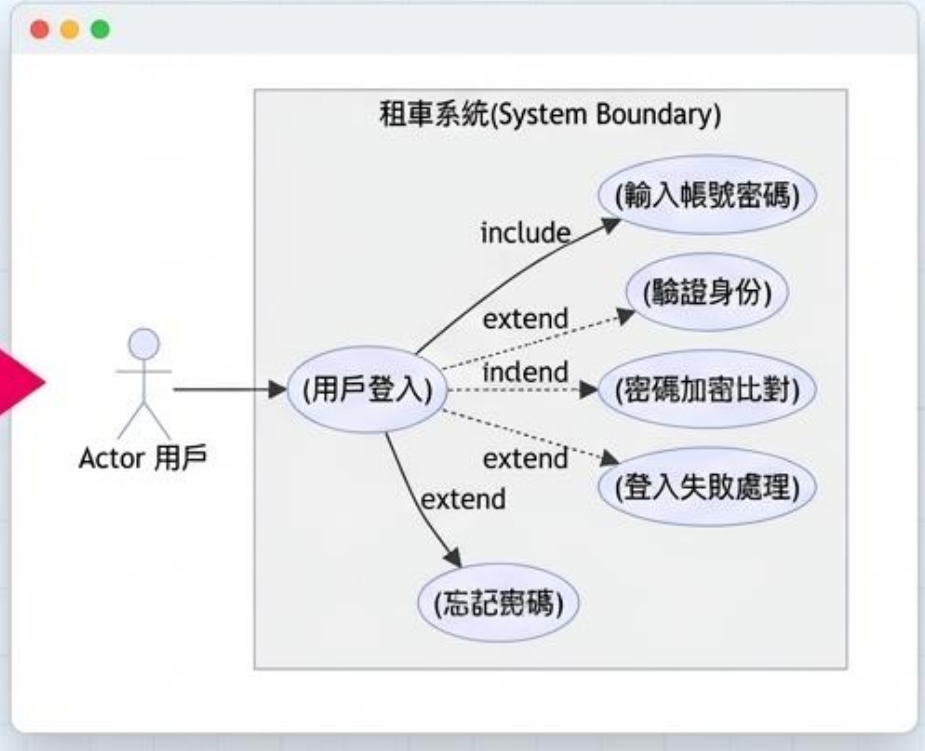
確立系統邊界

強制產生 subgraph，在架構圖上完美劃分內部系統範圍與外部 Actor。

步驟三：呼叫 Skill，瞬間生成邊界模型

```
/create-use-case-modeling
請依照用戶登入流程，將 Use Case 繪製在
#file:UC_01_用戶登入功能.md
```

Zero-touch Generation

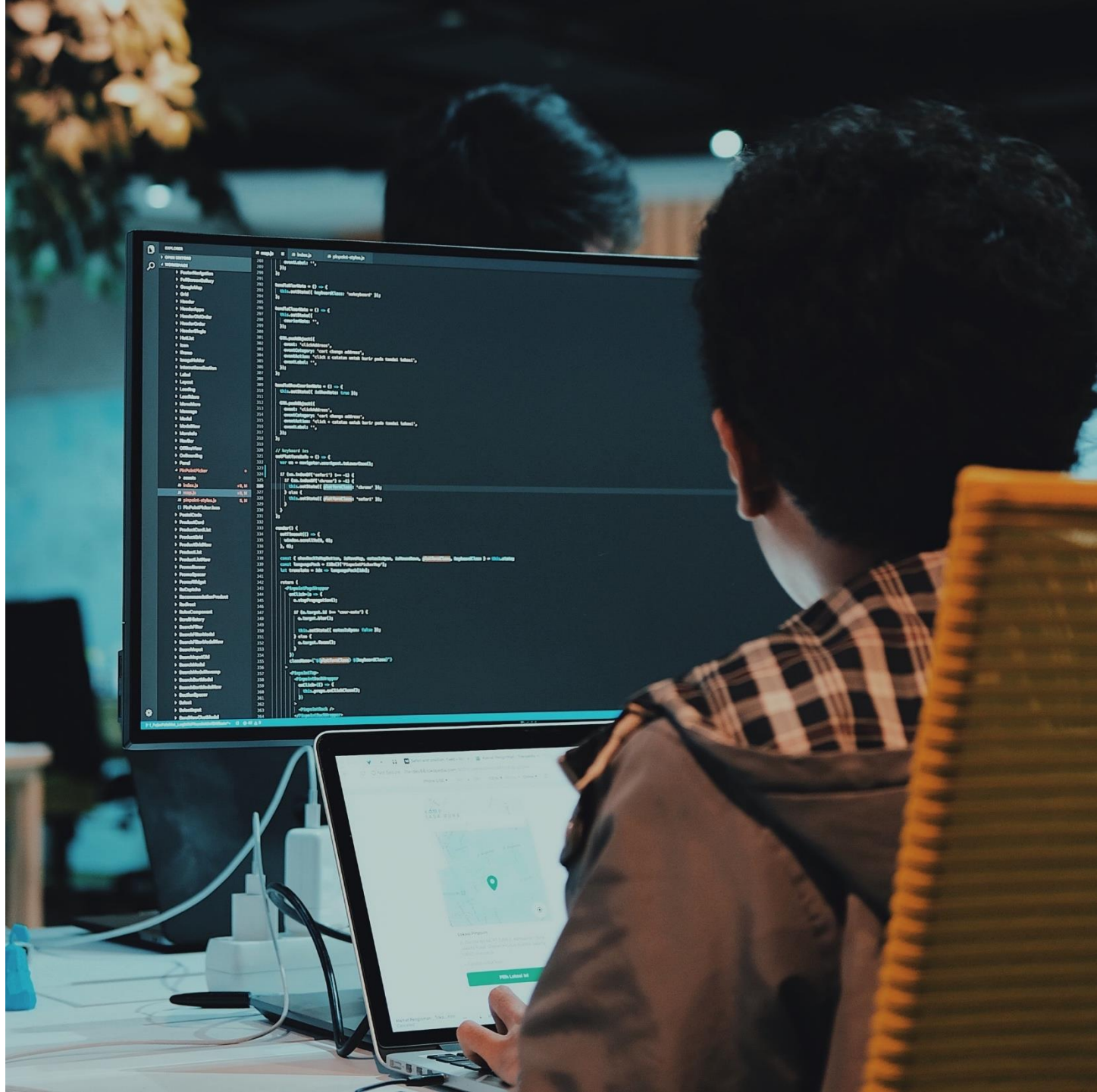


執行動作：
結合 `user-login.prompt` (業務邏輯) 與 `create-use-case-modeling` (繪製準則 Skill)。

卓越成果：
AI 嚴格按照指令產出 W3C 規範的 Mermaid 語法。無語法錯誤、邊界清晰，完全無需人工除錯。

Lab 1：流程 & 方法展示

(1).由 AI 展示 use case model到 sequence 到 domain class diagram 的設計過程



這就是為什麼我說，
重點還是在方法論？

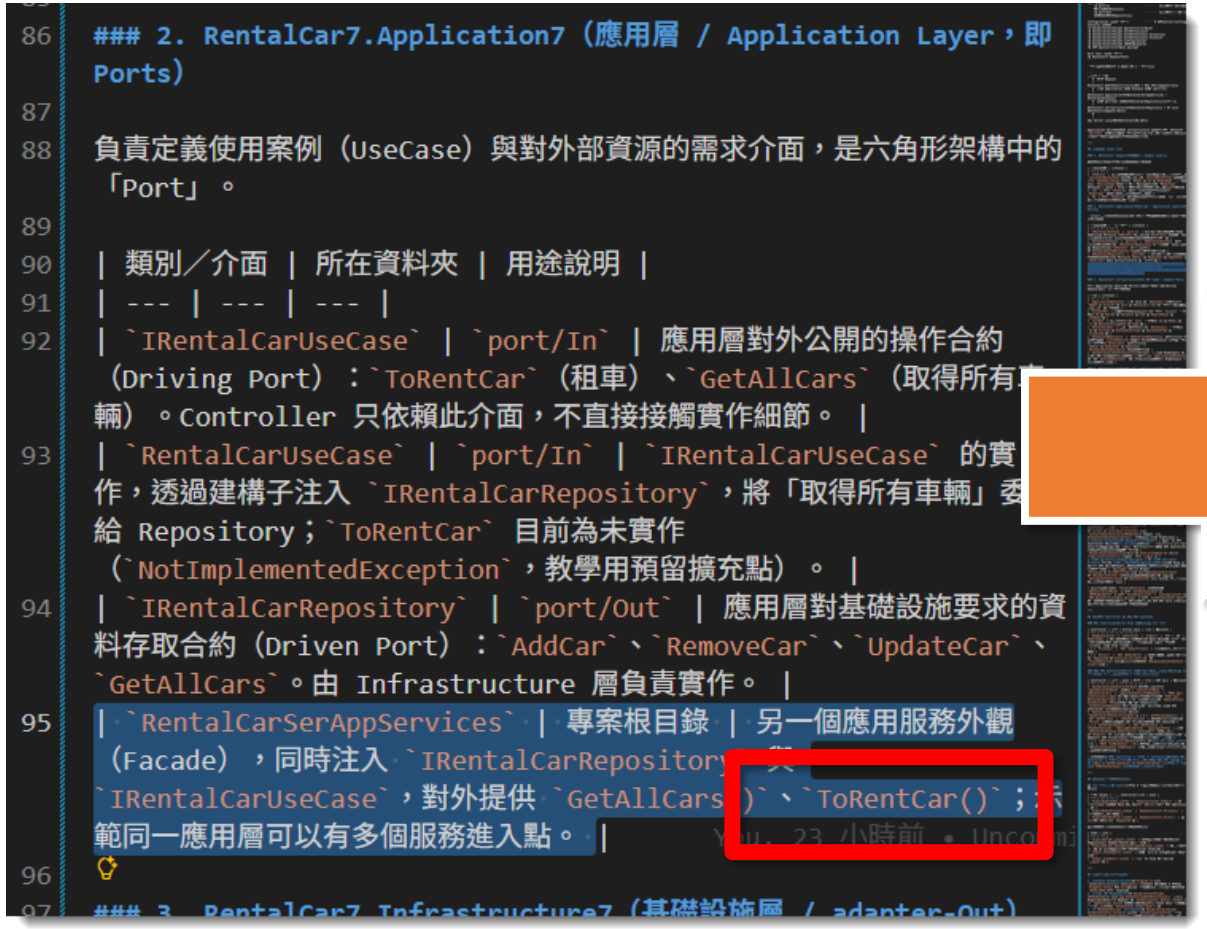
Methodology



https://github.com/wugelis/EasyV2LabForAI4

GitHub Copilot Claude

調整 ToRentalCar 方法的 SPEC





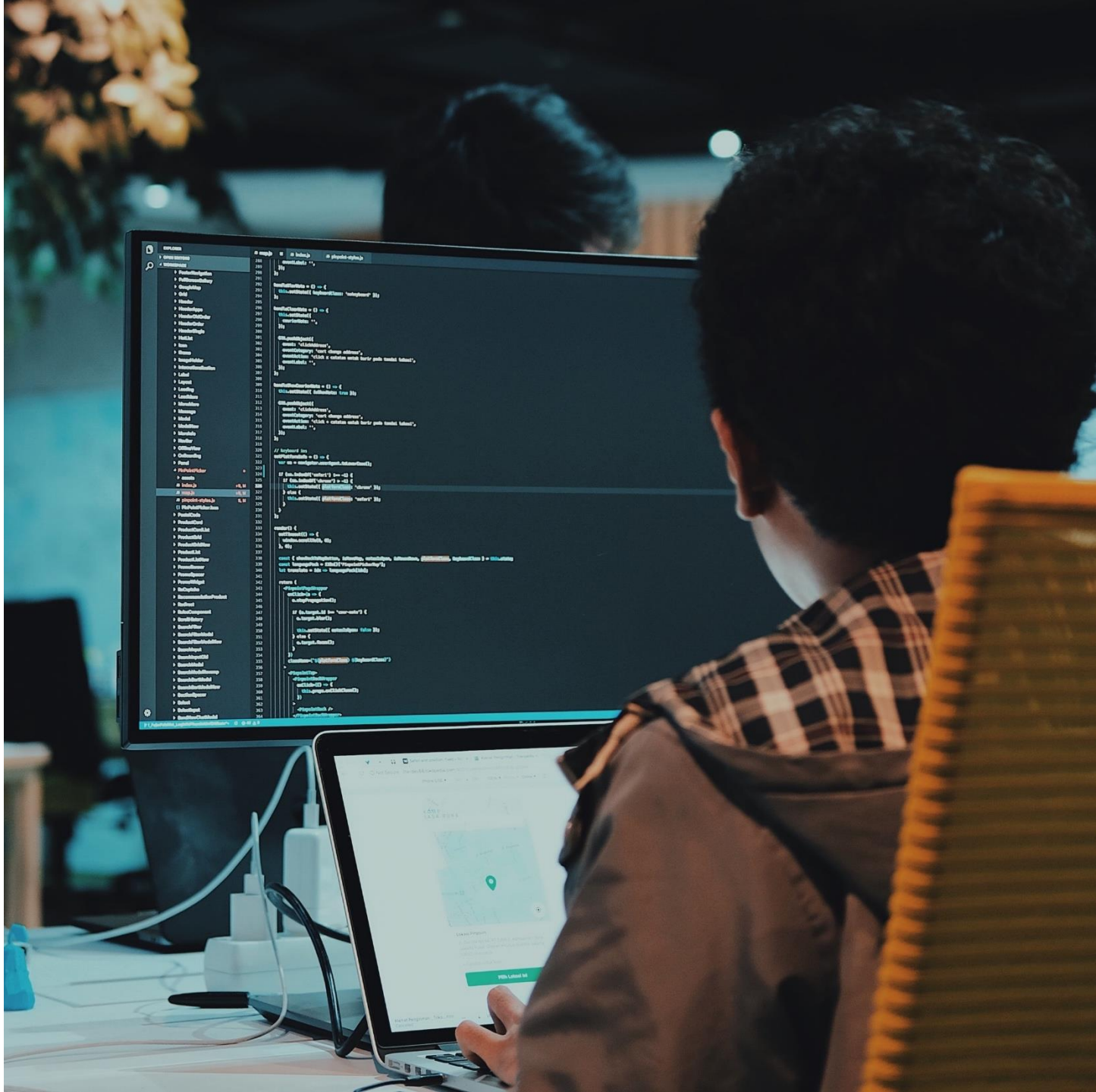
EasyV2Lab9

https://github.com/wugelis/EasyV2LabForAI4

Lab 2：建立 ToRentalCar(RentalCarDTO) 方法

(1). 完成前面投影片所有工作後
(2). 開啟新的 Claude 對話 Session
(3). 使用下面 prompt：

/application-rentalcar-rules
請參考增加的 ToRentalCar (RentalCarDTO) 規則，並在 @Application9.RentalCar9/RentalCarSerAppServices.cs 裡撰寫這個 C# Method @Domain9.RentalCar9/IVehicle.cs




Q&A

關於我



FB 社團
軟體開發之路



粉絲團
Gelis 程式設計訓練營



部落格
Gelis 技術隨筆-部落格

工商服務


HiSKIO
徵才中!
探索
搜尋
電子書抵用券
任務板
我的學習
首頁 › 微軟技術 | C#
C# 單元測試與 Moq 物件快速上手
從入門到實務，詳細講解專有名詞和艱澀的物件導向技術，告訴你如何最有效率的撰寫單元測試，讓你快速掌握精隨，輕鬆進入 CI 持續整合的門檻！
C#單元測試與Moq物件
快速上手
00:00 / 00:00
試閱課程
介紹影片
第 1 章 第一小節、為什麼要撰寫測試？與撰寫單元測試的好處？
(1). 撰寫單元測試的好處 23:17
第 2 章 第二小節、Unit Test 概論，從軟體開發常見問題探討 Unit Test 重要性
哪一種程式碼值得相信？與 單元測試基本定義與原則 11:52
第 3 章 第三小節、單元測試須具備的特性
F. I. R. S. T. 單元測試須具備的特性 04:43
優惠倒數 383 天 06 小時 17 分 13 秒

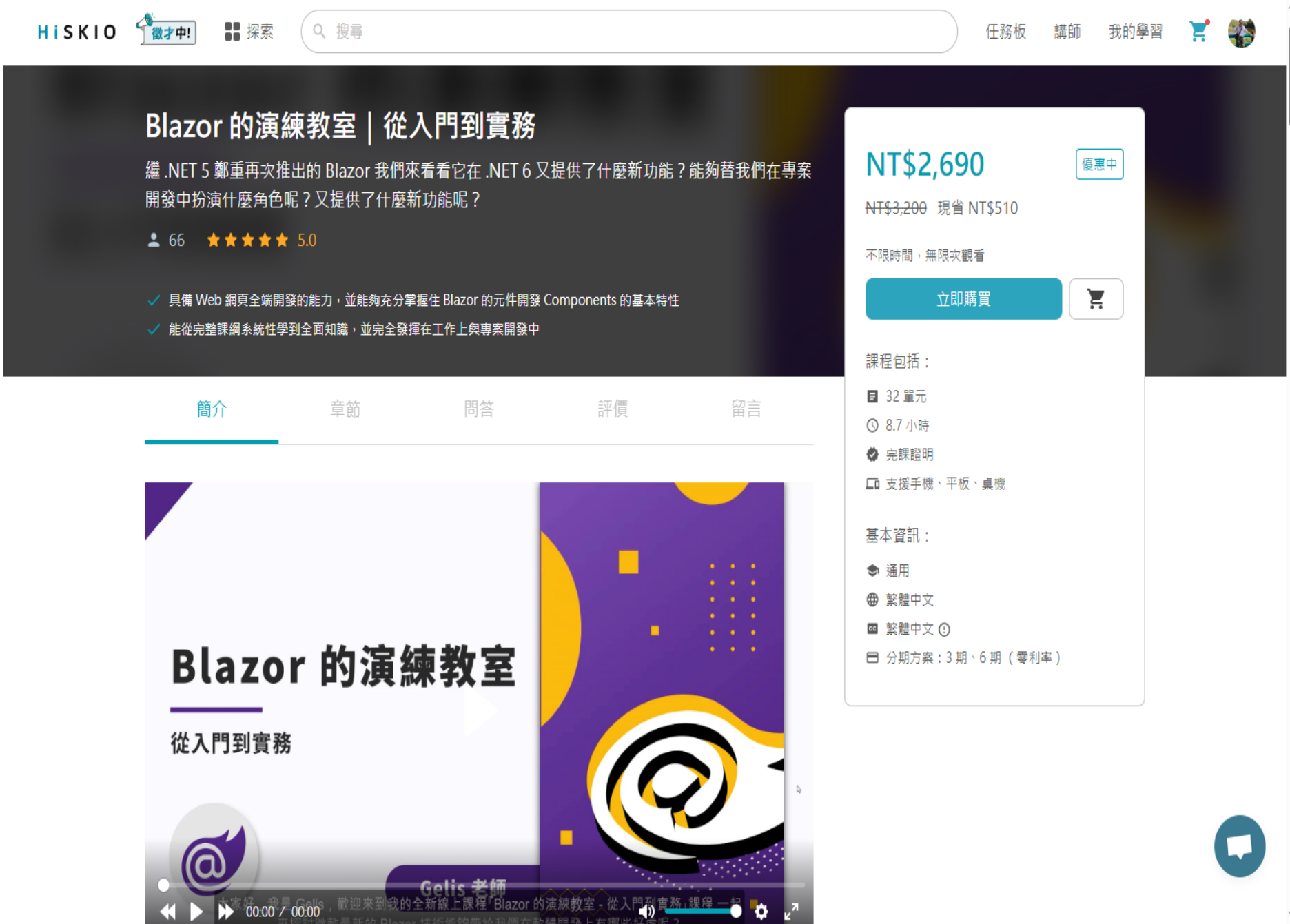
工商服務


HiSKIO
徵才中!
探索
搜尋
電子書抵用券
任務板
我的學習
首頁 › 微軟技術 | C#
C# Project Template 快速開發
學習透過 Visual Studio C# Project Templates 建立慣用開發方案！讓開發工作更加有效率 ，Visual Studio 2017 or 2019 Community/Professional/Enterprise 皆適用
Project
00:00 / 00:00
試閱課程
介紹影片
第 1 章 為什麼要製作 Project Template？與 製作 Project Template 可以達到什麼好處？
Visual Studio C# Project Template 課程目標、內容說明 05:02
什麼是 C# Project Template？它可以幫助您做到什麼？ 17:45
第 2 章 製作第一個Project Template
安裝 Visual Studio 擴充功能開發 與 將你的共用 CommonUtility 包裝成... 18:37
第 5 章 如何建置一個支援 IWizard 的 Project Templates？
5.2. 簽署 VSIX Assembly & 允許 VSIX 包含 Assembly 07:15
優惠名額有限，售完為止！別再猶豫了，馬上開始你的學習之路吧！

工商服務


HiSKIO
徵才中!
探索
搜尋
電子書抵用券
任務板
我的學習
首頁 > 電腦科學 | 設計模式 UML OOAD
決戰 OOAD 課程-系統分析設計與實作
在軟體技術蓬勃發展的現代，各種流程 Process 發展如雨後春筍般不斷湧現，像是：Agile / Scrum / Kenban 等等、以及您是不是曾聽說導入 Scrum 後，UML 就沒有用了的說法？誰說 UML 繪製的圖形都無法串到程式碼的實作的？本課程希望用比較新穎的 UML 分析概念帶大家進入一個不一樣的 UML 系統分析設計到實作殿堂。
決戰 OOAD
系統分析設計與實作
00:00 / 00:00
試閱課程
介紹影片
第 1 章 第一堂、近代軟體開發 與 UML
02小節-1、物件導向系統分析與設計概論 (OOAD) 、為什麼要使用 Mod... 05:04
第 2 章 第二堂、(扎穩馬步) 從身為UML兩大重心的「使用案例圖」與「類別圖」來探討分析概念
03小節、從身為UML兩大重心的「使用案例圖」與「類別圖」來探討分... 14:58
第 3 章 第三堂、UML 塑模化系統分析進階技巧
04小節、進行使用案例分析時該注意的幾個警訊 15:03
第 4 章 第四堂、實作 - （使用前面課程所產生的
優惠倒數 18 天 06 小時 06 分 24 秒

工商服務


HiSKIO
徵才中!
探索
搜尋
任務板
講師
我的學習
Blazor 的演練教室｜從入門到實務
繼 .NET 5 鄭重再次推出的 Blazor 我們來看看它在 .NET 6 又提供了什麼新功能？能夠替我們在專案開發中扮演什麼角色呢？又提供了什麼新功能呢？
66
5.0
具備 Web 網頁全端開發的能力，並能夠充分掌握住 Blazor 的元件開發 Components 的基本特性
能從完整課綱系統性學到全面知識，並完全發揮在工作上與專案開發中
NT$2,690
優惠中
NT$3,200 現省 NT$510
不限時間，無限次觀看
立即購買
課程包括：
32 單元
8.7 小時
完課證明
支援手機、平板、桌機
基本資訊：
通用
繁體中文
繁體中文
分期方案：3 期、6 期（零利率）
簡介
章節
問答
評價
留言
Blazor 的演練教室
從入門到實務
Gelis 老師
00:00 / 00:00